

Unbounded Data and Message Brokers

 systemdesignschool.io/fundamentals/stream-processing-intro



What is Stream Processing?

We've covered batch processing—scheduled jobs that process bounded datasets. But some applications can't wait. A fraud detection system that runs nightly analyzes yesterday's transactions and flags suspicious accounts by morning. But a fraudster who steals credit card data at 9 AM has all day to drain accounts. By the time the batch job runs, the money is gone.

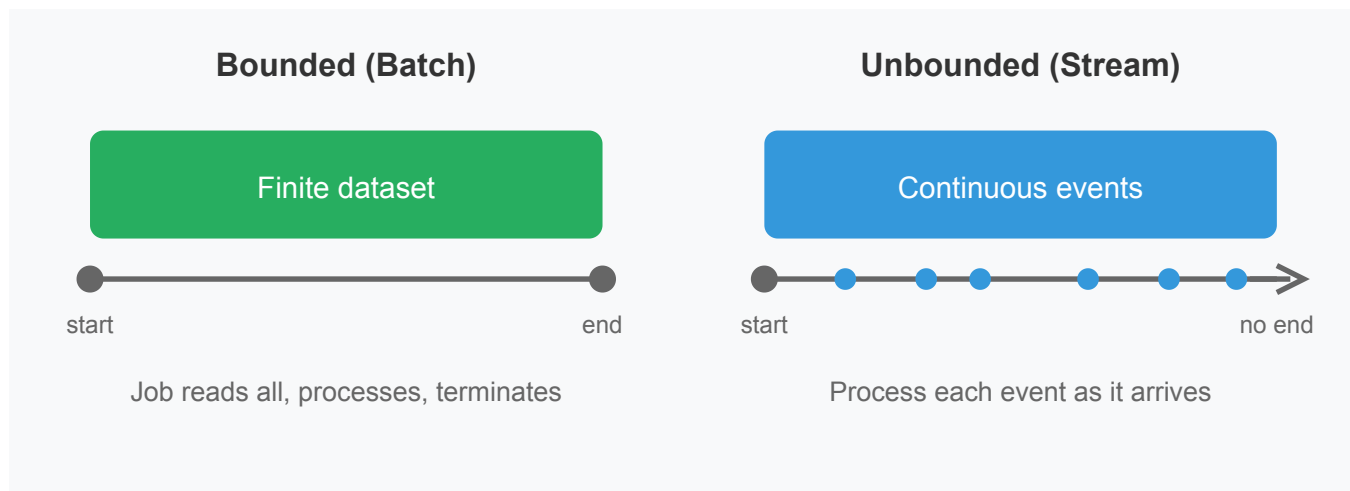
Stream processing takes a different approach: handle data continuously as it arrives, rather than waiting for it to accumulate.

Bounded vs Unbounded Data

The fundamental distinction between batch and stream processing is the nature of the input.

Batch processing works with **bounded data**—a finite dataset with a defined beginning and end. You might process "yesterday's log files" or "all transactions from Q3." The batch job reads all input, processes it, and terminates.

Stream processing works with **unbounded data**—a continuous flow of events with no defined end. Events keep arriving: user clicks, sensor readings, financial transactions. The processing system runs indefinitely, handling each event as it arrives.

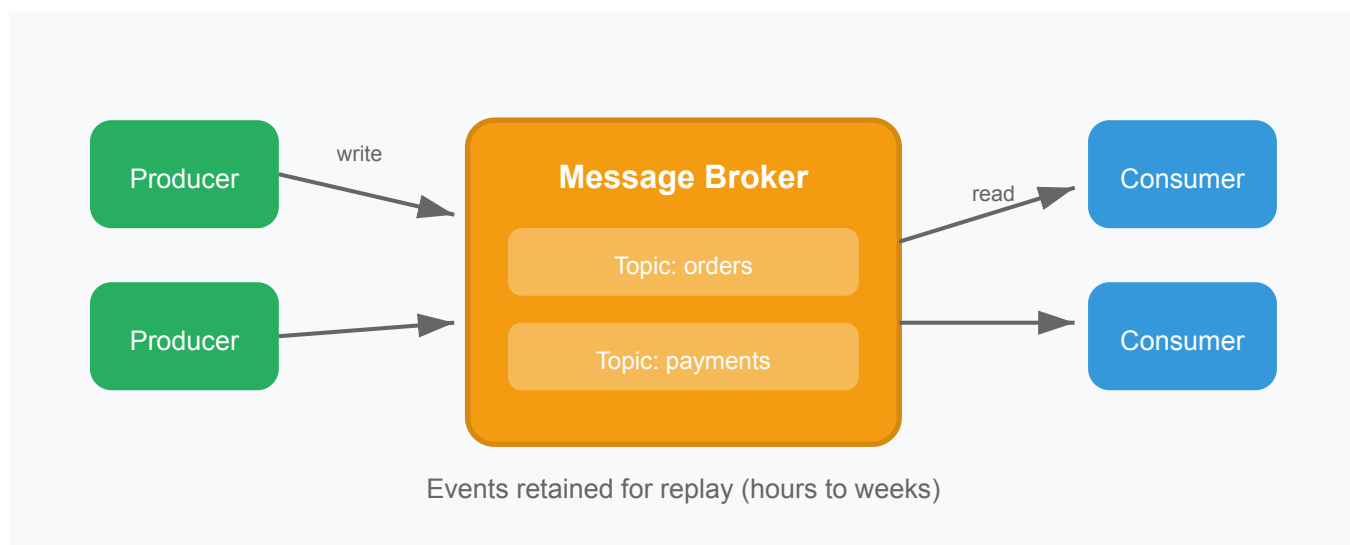


Think of it like the difference between counting fish in a bucket versus counting fish swimming past a point in a river. The bucket has a fixed number; you count them all and you're done. The river never stops—you count continuously and report rolling totals.

Message Brokers: The Input Layer

Where does stream data come from? The standard architecture uses a **message broker** (sometimes called an event bus or streaming platform). Apache Kafka is the most widely used, though alternatives include AWS Kinesis, Apache Pulsar, and others.

Producers (your application services) write events to **topics**—named categories like "user-signups," "order-placed," or "sensor-readings." Consumers (stream processing applications) subscribe to topics and receive events in order.



The broker provides key capabilities:

Decoupling. Producers don't need to know which consumers exist. They write events; consumers independently subscribe to the topics they care about.

Durability. Events persist in the broker for a configurable retention period—hours, days, or indefinitely. If a consumer crashes, it can resume from where it left off. If you deploy a new consumer, it can replay historical events.

Ordering. Within a partition (a subset of a topic), events maintain their order. A consumer sees events in the sequence they were produced.

Scalability. Topics partition across multiple brokers. Consumers parallelize by assigning different partitions to different instances.

The Stream Processing Pattern

A stream processing application follows a simple loop:

1. **Read events** from input topics
2. **Process each event**—filter, transform, enrich, aggregate
3. **Produce output**—write to output topics, update databases, trigger alerts

The application runs continuously. Unlike batch jobs that start, process, and terminate, a stream processor is always running, waiting for the next event.

State is the tricky part. Many stream operations need to remember information across events. Counting page views requires remembering the running count. Detecting "three failed logins in 5 minutes" requires remembering recent login attempts. Stream processors maintain this state, often backed by embedded databases or external stores for durability.

Key Differences from Batch

Aspect	Batch	Stream
Input	Finite dataset	Continuous events
Execution	Start, process, terminate	Runs indefinitely
Result latency	Minutes to hours	Seconds to sub-second (typically)
State	Often rebuilt each run	Maintained continuously
Debugging	Examine frozen input	Data keeps flowing

When Stream Processing Fits

Real-time decisions. Fraud scoring must happen before a transaction completes. Recommendation engines should respond while the user is browsing. Waiting for a batch job means missing the window.

Live dashboards and monitoring. Operations teams need current metrics: requests per second, error rates, active users. Hourly batch updates aren't actionable for incident response.

Event-driven architectures. An order placement triggers inventory reservation, triggers payment processing, triggers shipping notification. Each step produces events that the next step consumes. The system reacts immediately rather than waiting for scheduled jobs.

Alerting on thresholds. When error rates exceed 5%, alert the on-call engineer now—not when the nightly batch job discovers it.

What's Ahead

Stream processing introduces challenges that batch processing doesn't face. When data arrives continuously, how do you aggregate "per hour" when the stream never ends? That leads into **windowing patterns**—tumbling, sliding, and session windows that let you compute bounded aggregates over unbounded data.

After that, we'll tackle event time versus processing time (what happens when events arrive late?), and then delivery guarantees (how do you ensure every event is processed exactly once?).